Ref. Ares(2026)9138649 - 28/09/2026





One-Stop-Shop Artificial Intelligence and Machine Learning for Official Statistics

Project 101146355 – AIML4OS

Workpackage 12

Use Case: Large language models

Deliverable	12.2 – WP12 LLM prototype B
Month due	Sep 2026
Type	DEM — Demonstrator, pilot, prototype R — Report
Prepared by	Jakob Engdahl (Statistics Sweden)

Workpackage Leader:

NAME: Jakob Engdahl
Jakob.engdahl@scb.se

Disclaimer: Views and opinions expressed are those of the author(s) only and do not necessarily reflect those of the European Union or Eurostat. Neither the European Union nor the granting authority can be held responsible for them.

Table of contents

1. Introduction 3
2. Composition of this deliverable 4
3. Approach to evaluation 5
4. Architecture and infrastructure 6
5. The prototypes and the experiment 8
5.1 Web Corner: agentic collection of web data 8
5.2 Metadata Graph: artificial intelligence support for metadata management 10
5.3 News Corner: consistency between media reporting and official releases 13
6. Evaluation results 15
7. Scope and limitations 16
8. Observations on the quality of generative approaches 16
9. Future work 17
10. Conclusions 18
Supporting material and information 20

1. Introduction

Work Package 12 (WP12) of the AIML4OS project examines how large language models can create value for statistical organisations in Europe. The work combines practical prototype development with reflection on how such systems can be introduced in a way that supports reuse, data protection, transparency, evaluation and operational relevance.

The work package proceeds from the observation that large language models already offer substantial opportunities in the statistical domain, including text automation, code translation, quality control and conversational interfaces, while the pace of development makes it difficult to forecast which specific applications will prove relevant over the period 2024 to 2027. WP12 therefore identifies five presumptive high-value areas and works iteratively, selecting concrete applications as late as possible before each piece of work begins. The five areas are the handling of data and metadata through the use of large language models; the generation of draft text for the Analyse phase of the statistical production process; the improvement and translation of production code; the use of conversational interfaces in dissemination; and the analysis of large documents and web page data.

Across these areas the work package pursues three objectives. The first is to explore the ability of the models available during the project period to integrate into the production and support of official statistics. The second is to explore what benefits fine-tuning brings to the identified use cases. The third, which runs through both of the others, is to define the architectural enablers and constraints that apply when such models are used.

The work package is organised in three tasks. Task 12.1 analyses how large language models can be integrated into production in ways that align data protection with the sensitivity of the data concerned, and is to produce architectural guidance supporting the prototype work. Task 12.2 demonstrates the use of pre-existing models through at least two prototypes covering at least two of the high-value areas. Task 12.3 examines fine-tuning as a route towards a specialised statistical language model.

This deliverable is the second prototype deliverable under Task 12.2. Its purpose is to demonstrate what existing, unmodified models can do for official statistics, and to record what was learned in enough detail that other national statistical institutes can judge whether to pursue similar work themselves.

It should be noted that Task 12.1 has not yet produced guidance documents or other material. The collection of input for that work has begun, but nothing in this report should be read as applying or summarising established architectural guidance. The architectural observations and the evaluation criteria set out below run in the opposite direction. They are an indication, drawn from building and operating the prototypes, of what may be worth having in mind when an architecture or design for artificial intelligence solutions in these application areas is developed, and they are offered as input to Task 12.1 rather than as output from it.

The material behind this report was developed over an extended period through several activities, including collaborative development sessions, independent work by the participating organisations between those sessions, and testing carried out in the shared development environment provided through Work Package 3. A three-day development sprint hosted by Statistics Sweden in June 2026 was one of these occasions and produced a substantial part of the technical material, but it was not the only source of the knowledge and experience recorded here.

Deliverable D12.1 documented three early prototypes concerned with the summarisation of published reports, the extraction of structured data from documents, and the classification of web content. In all three, the language model was used to interpret material that conventional software had already retrieved. The model acted on text placed in front of it.

The work reported here continues two of those threads and adds a third area, and the substantive difference lies less in the subject matter than in the architectural level of the pilots. In the present prototypes the model itself decides what to retrieve or which relationships to follow, and the surrounding system provides it with the means to act rather than with a finished body of text. Three techniques that were not used in D12.1 have now been applied in working systems: tool calling, by which a model invokes defined operations and receives their results before deciding what to do next; the Model Context Protocol, an open standard by which a system can make its operations available to any compatible agent; and the packaging of domain knowledge as loadable skills that shape and constrain how a model behaves in a given professional context.

The pilots are consequently more advanced in architectural terms than those reported in D12.1. They are no longer single exchanges with a model but systems in which the model participates in a controlled loop, and in one case a system that other agents can address directly.

2. Composition of this deliverable

The deliverable consists of a report and a prototype. In practice the prototype requirement is met by two technical prototypes, each demonstrating different properties and architectural approaches to generative artificial intelligence, complemented by a report on an experiment covering a third use case.

The first prototype, referred to as Web Corner, is an agentic web scraper. It falls within the high-value area concerned with the analysis of large documents and web page data. It demonstrates how much capability can be obtained from a very small system when the reasoning is carried out by the model: the application consists of a short control loop and a handful of narrow operations, and its behaviour is determined largely by the instructions it is given rather than by its code.

The second prototype, referred to as the Metadata Graph, is a knowledge graph for statistical metadata with a conversational interface. It falls within the high-value area concerned with the handling of data and metadata. It demonstrates what is required when the capability must be embedded in an organisation's information model, governed by a schema, and made available to other systems.

The use cases were selected on a common principle. Each represents a task that occurs routinely in a statistical organisation, and each is a task for which conventional rule-based systems either fall short or achieve the result only at a level of ongoing maintenance that is difficult to justify. Collecting information from websites that change their structure, and keeping a metadata system usable by people who are not metadata specialists, are both of this kind. The two prototypes that emerged complement one another well: one shows how much can be achieved with a very small system when the reasoning is left to the model, the other what is required when the capability has to be embedded in an organisation's information model and made available to other systems.

The third contribution, referred to as News Corner, concerns the consistency between media reporting and official statistical releases. This use case is becoming steadily more relevant for statistical organisations, which may need both awareness of how their output is being reported and

the means to counter inaccurate representation of the figures they produce. It is also of interest because it bears directly on the multilingual requirements of the European Statistical System: the practicality of this kind of analysis depends on the language model being able to work across the wide range of languages used within the system, and not only in English.

This third contribution is reported as an experiment rather than as a prototype. The work was deliberately set up in a simple way, with the emphasis not on the architecture but on showing how tests can be constructed to probe what a language model is capable of in relation to an actual operational need. More elaborate solutions can follow to the extent that continued development is judged worthwhile. It is included here because the test evidence is directly useful to any organisation considering the same task, and because the architecture it points towards is described in enough detail to be implemented by others.

Fine-tuning and domain adaptation fall under Task 12.3 and are not attempted here. There is no requirement for a single deliverable to address all five high-value areas, and the remaining three are outside the scope of this one.

3. Approach to evaluation

To support the sustainability and maintainability of the systems developed in the project, each use case is assessed along a common set of dimensions: the efficiency gain relative to manual or existing approaches; reusability by other organisations; the accessibility of the required input data; compatibility with on-premises operation; the ease of adoption by an office using its existing infrastructure; the robustness with which output quality can be assessed; technical feasibility within realistic resource constraints; expected useful lifespan given evolving technology; and cost effectiveness in relation to the value delivered.

It is useful to separate two questions that the word evaluation covers, because this report reaches only the first of them.

The first question is whether a system is reasonable: whether it can be built, run, reused and operated under realistic conditions, and whether it respects sensible architectural principles. That is what the dimensions above address, and it is what this report evaluates. The criteria themselves were formulated during the work rather than derived from finished guidance, and should be read as an indication of the dimensions that appear to matter when designing solutions of this kind, not as a settled framework.

The second question is whether a system is effective: how accurate, reliable and useful its output actually is when measured systematically against a known correct answer. The work so far has not had the preconditions for answering this. The shared environment used for development permits only non-sensitive material, no benchmark datasets existed to measure against, and the available development time was limited. Where this report comments on output quality it therefore reports observations and individual test cases rather than systematic measurement.

That distinction defines what the prototypes are most useful for. Their value is precisely that they make the second kind of evaluation possible. They are working systems that can serve as the basis for systematic assessment of effectiveness, carried out in environments where sensitive material may also be used and where an organisation's own data can supply the reference against which output is judged.

One further note concerns how the assessment of on-premises compatibility should be read. All three use cases are architecturally compatible with on-premises operation, since none depends on a particular supplier and each communicates with a language model through a replaceable, standard interface. The assessment therefore reflects what an office must provide locally for the system to work well, rather than whether the software can be directed at a local endpoint. Two of the three require a model that supports tool calling, which in practice means operating a model of moderate size on local hardware. That is a genuine prerequisite and the main reason those two are assessed as medium to high rather than high.

4. Architecture and infrastructure

All three use cases were developed and tested in the SSPCloud environment, and all three were exercised against the same open-weights model served from that environment, with supplementary comparisons against a commercial service in one case.

SSPCloud is in practice a cloud service available to national statistical institutes, operated by INSEE, the French statistical office. It carries the conditions that such a service brings with it, among them a requirement that only non-sensitive material may be handled in the environment. All the work reported here therefore used public or otherwise non-sensitive data, and nothing in this report demonstrates that these systems may be used on sensitive statistical material. Establishing that would require an environment cleared for the purpose, which is a separate question from whether the software runs.

What the arrangement does provide is a realistic picture of the conditions that many statistical offices could establish, either locally or within whatever arrangements are available nationally for using comparable services. The same holds for serving open models: an office able to operate a model of moderate size reproduces the essential property of this environment without depending on this particular service.

The three use cases selected here should also remain workable under that restriction as the work continues. Metadata is seldom regarded as sensitive material in itself, and the other two use cases draw on information that is already publicly available on the internet. There is, however, a drawback to relying on sources that sit on the internet rather than on material that could be packaged as part of the deliverable. Content published on the web is continually updated and reorganised, which makes it difficult to maintain these prototypes against a fixed set of references, whether those are links to particular news articles or example pages used to exercise agentic collection. A demonstration that works today may fail some months later for reasons that have nothing to do with the system itself.

That all three use cases ran against the same model on the same infrastructure is itself a result, since it means the differences observed between them reflect the tasks and the architectures rather than differences in access to models.

Three questions dominate the architectural picture that emerges, and the prototypes speak to each of them. On openness and practicality, the open-weights model proved sufficient for every task attempted, and was outperformed by a commercial model only on the single most difficult comparison case, where the difference was resolved by revising the instructions given to the model rather than by changing model. Choosing an open model therefore did not cost capability at this level of ambition. On reusability, both prototypes reach their model through a standard interface, so

the choice between the shared service, a locally operated model and a commercial service is a matter of configuration rather than of code. This is the property that allows the same system to be adopted by offices working under different constraints. On data sensitivity, the relevant observation concerns the shape of the solution rather than the material processed: because the model endpoint is replaceable, and because the Metadata Graph operates fully without any model configured at all, an office can place the boundary between its own data and any external service wherever its rules require, and can deploy the parts of the system that do not involve a model before taking any decision about one. Whether a particular deployment is then permitted to handle sensitive material is a matter for that office's own assessment.

Taken as a whole, the prototypes indicate that for several application areas within official statistics it is possible to establish solutions built on the strengths of generative artificial intelligence under conditions that a statistical office can realistically meet. The cost of that portability is the infrastructure prerequisite noted above: a model capable of tool calling still has to be operated somewhere.

Both prototypes are built on tool calling, but they apply it in structurally different ways, and the contrast is one of the more transferable architectural findings of the work.

In the first pattern, illustrated by Web Corner, the agent owns its operations. A small set of narrowly defined functions is offered to the model with each request, and a short loop performs whatever the model asks for, returns the result and asks again. The operations are private to the application and nothing outside it can invoke them. This is the lightest agentic architecture available and it suits a bounded, self-contained task.

In the second pattern, illustrated by the Metadata Graph, the same underlying capability is instead published through the Model Context Protocol. This is an open standard that defines how a language model discovers and invokes external operations and data sources. It plays the same role for the integration of agents and systems that established protocols play elsewhere in software: it provides a common contract and removes the need for bespoke wiring between each agent and each system. In the Metadata Graph the graph operations are offered on two parallel interfaces backed by the same underlying service, one intended for a person working in a browser and one intended for a language model operating on its own behalf. Both run within the same application, with no separate infrastructure.

Three properties of that arrangement generalise beyond this particular system. First, what an external agent is told about the domain is derived from the deployment configuration rather than written into the code, so adapting the system to another organisation's information model and briefing an agent correctly become the same act. Second, the human interface and the agent interface are backed by a single service layer, so what an agent can see and change is exactly what the application permits a person to see and change; this is a governance property as much as a technical one. Third, the effect is a change in kind rather than degree: a system reachable over a standard protocol ceases to be an application with an artificial intelligence feature and becomes a source that other agents can reason over. In practical terms, an office operating the Metadata Graph can direct an external assistant at its own metadata and query it conversationally without any integration work.

This second pattern is also the prerequisite for the next mode of operation. Both prototypes currently run bounded loops driven by a single request from a user. Autonomous agentic loops, in which an agent plans, traverses, evaluates intermediate results and decides whether to continue,

retrace or conclude, are what several of the metadata use cases actually require. Assessing the consequences of a change to a statistical classification cannot be answered in a single exchange, because the queries that need to be issued depend on what earlier queries returned. The architecture does not yet implement that loop, but it deliberately provides the structure within which it can be built.

Placed in the terms current in the field, these capabilities are steps towards agentic systems and, further out, towards autonomous agents. The point of that development is not only to establish that a given task can be performed. It is to make use of a language model's capacity to reason in order to build systems that are more flexible than their rule-based predecessors, and in particular systems whose maintenance cost does not rise in step with every change to the sources they depend on. The argument made earlier about collection from websites is the clearest instance: what previously required a script per site is carried instead by a set of instructions and a small number of general operations.

The arrangement whereby a system offers comparable access to human users and to artificial intelligence agents belongs to the same development. It is a practical route towards what is increasingly described as artificial-intelligence-ready data and metadata, in the sense that the holding is not merely published in a form that a person can read but exposed in a form that an agent can discover, interpret and act upon without an integration project being undertaken first.

5. The prototypes and the experiment

This section describes each of the three contributions in turn: what it does, how it operates, what was tested and what the results were, together with a note on how a reader can take it further. The assessment of each against the common dimensions follows in Section 6.

5.1 Web Corner: agentic collection of web data

Traditional collection of data from websites relies on rule-based scripts written for a particular site, which fail whenever that site changes its design. Web Corner replaces such scripts with an agent that interprets a request expressed in ordinary language, navigates the pages itself, and decides where to look for the information sought.

The system is given a natural language objective and, optionally, a starting address. An example of such an objective would be a request to find all job listings related to information security on a named statistical office's website. The output is a structured answer containing the information requested, the individual addresses at which it was found, and a record of the reasoning by which the agent arrived there.

The system operates as a decision and action loop. The model evaluates the context of the page it is currently considering and decides what to do next. The application performs the requested action and returns the result. The model then determines whether the task is complete or whether further navigation is required, and the cycle repeats until it produces an answer rather than a further request.

Three operations are available to the model. It can obtain the list of links present on a page, which allows it to move from a listing page to the individual pages it points to. It can obtain the readable text of a page, with the underlying markup removed. And it can interact with a page by clicking, entering text or scrolling, which is necessary on sites where content only appears in response to such

actions. All three operate on a fully rendered page, so material that is only assembled when the page is displayed is available to the model in the same way that it would be to a person.

Two design choices carry most of the system's behaviour. The operations are deliberately narrow, each performing a single small task, which makes the model's use of them considerably more reliable than when it is offered composite operations. And the control loop is deliberately thin: it conveys requests and results and does nothing else, so all navigation strategy resides in the instructions given to the model rather than in the code. Those instructions distinguish listing pages from the individual pages that hold the actual information, require the agent to follow pagination rather than settle for the first page of results, and identify as failures the two most common wrong outcomes, namely returning the address of a directory instead of the item sought, and reading the text of a listing page instead of the individual pages it points to. Because the strategy resides in the instructions, a new collection task can be set up without writing anything.

Three tasks were carried out and their full transcripts retained. The first requested the most recent inflation figures from the Swedish and Dutch statistical offices, presented as a table; the agent returned both figures with their reference periods and the addresses of the releases they came from. The second requested the vacancies at the Dutch statistical office that involve fieldwork rather than office work, with the address of each; the agent returned five individual vacancies with their addresses. The third requested the addresses, prices and review sentiment for a product category on a commercial retail site; the agent returned fifteen products in a structured format.

Each of the three reached the level of the individual item rather than stopping at a listing page. The vacancy task additionally exercised the system's ability to locate a starting point on its own, since it was given no starting address and found the relevant recruitment site itself. The three tasks consumed between thirteen thousand and twenty-four thousand tokens in total, including the model's reasoning, which places the cost of a completed task at a few cents when a commercial service is used and at compute alone when the model is operated locally.

The operations that retrieve pages present themselves to the target site as an ordinary desktop browser rather than as an automated client. This is what allows the system to work on sites that block automated access, but it is an active measure to avoid detection rather than a neutral technical choice. An office adopting this approach should decide deliberately whether it is compatible with its own policy on automated collection, with the terms of use of the sites concerned, and with the transparency expectations that apply to official statistics. This is a property of the agentic collection pattern rather than a defect of this particular implementation. It is one of the questions that the architectural work under Task 12.1 will need to describe in greater detail, alongside data protection, since guidance on agentic collection will have to state what is acceptable practice as well as what is technically possible.

The retrieval, extraction and reasoning functions are fully operational. The system can render pages whose content is assembled dynamically, but the coordination between the model's decisions and the timing of that assembly is still being refined, and reliability on the most dynamic sites is not yet assured. Automated comparison against standard datasets has not been started.

A reader wishing to try the system can obtain it from the work package repository, where it is published together with its own report, the full transcripts of the three runs described above, and instructions for setting it up. Two things are required. The first is an environment in which Python can be run, since setting the system up involves retrieving the code, creating a Python environment, installing its dependencies and the browser components it uses to render pages, and copying the

supplied configuration template. The second is access to a language model that supports tool calling and is reachable through a standard interface.

Both can be satisfied in more than one way. The shared environment provided through Work Package 3 and described in Section 4 offers a development workspace and a hosted model together, and the instructions explain how to obtain the necessary credentials for it; this is the least demanding route for a reader without local infrastructure. Equally, the system can be run on an ordinary workstation or server against a model operated by the organisation itself or obtained from a commercial service, which requires changing three values in the configuration file and nothing else. A task is then expressed as a short instruction in ordinary language, and the three worked examples supplied with the system serve as a starting point that can be adapted to a new collection task without any programming.

5.2 Metadata Graph: artificial intelligence support for metadata management

Metadata is a precondition for efficient and quality-assured statistical production, yet in many statistical organisations the metadata that exists serves mainly as documentation rather than being used actively to govern how statistics are produced. Metadata systems are frequently custom-built, and they are often among the more neglected parts of an organisation's infrastructure, in part because metadata is a specialist field that comparatively few people within a statistical organisation are trained in. This creates a practical difficulty: the producers of statistics and the metadata specialists who maintain the formal descriptions of that production do not always share a working vocabulary. Supporting the communication between them, and moving metadata from a passive record towards something active and usable by machines, is a direction that several statistical organisations are currently pursuing. That is the context in which this use case was chosen.

The Metadata Graph makes a statistical metadata system explorable visually and conversationally. Rather than navigating manually through registers and forms, a user sees concepts, variables, classifications, populations and process steps as connected nodes. Any element can be selected and asked about in ordinary language, and the answer is generated by a language model working from the actual content of the graph rather than from general knowledge. Users can also enrich the graph, adding concepts and relationships directly, with the model mediating between the user's own words and the formal expression of those concepts in the relevant standards.

The graph model is not tied to the information model of any particular organisation. It is deliberately flexible and configurable, so that it can be shaped to the range of models an organisation actually works with, among them the Generic Statistical Information Model (GSIM), Statistical Data and Metadata eXchange (SDMX), the Data Documentation Initiative (DDI) and Dublin Core, and so that the concepts it presents are recognisable to a metadata specialist. This is a matter of the vocabulary and structure being configurable to align with such standards; it does not mean that the system implements any of them as an exchange format or communicates in them directly.

The application consists of a graph and chat interface presented in a browser; a service that holds the graph, answers requests about it and calls the language model on behalf of the conversation; and a language model that is replaceable. Two properties of this arrangement matter for the present purposes. The first is that the application starts and operates with no language model configured at all, in which case the graph and its interfaces remain fully available and the conversational features

are simply absent. An organisation can therefore adopt the graph itself before taking any decision about a model. The second is that the domain model is entirely a matter of configuration. The node types, the permitted relationships between them and the domain context presented to the model are all defined in a deployment profile rather than in the code, so adapting the system to another organisation's information model is a configuration exercise rather than a development project.

Alongside the interface used by people, the graph operations are published through the Model Context Protocol, as described in Section 4. This is what allows an external assistant to be directed at an office's own metadata and to query it conversationally without any integration work.

A further capability distinguishes the system from a general-purpose combination of a graph and a conversational interface. Rather than relying on a single fixed instruction to the model, domain knowledge can be supplied as skills: structured documents carrying the established concepts, standard classifications and methodological conventions of a particular statistical domain, which are attached to expert agents that a user can activate. Skills are loaded progressively, so that the system knows at all times which skills exist while the content of a skill only enters the model's working context when a user is actually working in that domain. Each skill also declares which graph operations may be used while it is active, so it both supplies domain knowledge and constrains what the agent may do. Several skills can be active at once, and moving between them during a session does not interrupt the work.

The practical effect is that guidance offered during metadata curation can refer to the conventions of the organisation and the domain rather than to generic advice about metadata. A skill written for population statistics, for instance, might carry the established age and sex breakdowns and the recommended value domains for demographic data, so that a curator working in that area receives guidance grounded in those conventions. The profile built for this work ships two skills as worked examples, together with expert agents through which they are activated; one of these agents is deliberately minimal and serves to show how an organisation would define a narrow domain agent of its own rather than to supply substantive domain knowledge.

The underlying graph application is a general-purpose platform that existed before this work rather than being built for it. The contribution reported here was to apply that platform to official statistics, and in the course of doing so to develop it further where the statistical use cases called for capabilities the platform did not yet have or had only in outline. The work of applying it involved building a deployment profile for the European Statistical System, comprising the schema, the presentation, the expert agents and the domain knowledge skills; assembling a body of example metadata covering statistical offices, statistical programmes such as the Labour Force Survey, and the datasets, data structures, variables, concepts, unit types, code lists, questionnaires and classifications associated with them; defining a set of user stories describing the statistical use cases the system should support; and documenting how the system can be launched and explored in the shared environment.

This distinction matters when reading the assessment below. The judgements about reusability and architecture concern the platform, whereas the judgements about feasibility and coverage concern the profile and example data built on top of it.

The schema defined for this domain covers eighteen node types and twenty-seven kinds of relationship, modelled on GSIM. It includes the relationships needed to trace how one dataset is derived from another and the relationships needed to represent successive versions of a statistical classification, both of which are prerequisites for the use cases described below.

Six user stories were defined and four were exercised against the system. The first concerns extraction of concepts from documents. A statistical producer uploads a questionnaire or similar documentation, and the system identifies the concepts, variables and value domains it contains, compares them against the metadata already held in the graph, and proposes how new content might be represented and where existing structures could be reused. This was tested using a real labour force survey questionnaire.

The second concerns lineage and the assessment of the consequences of methodological change. A methodology officer selects a published dataset and follows its derivation chain; conversely, when a statistical classification moves to a new version, the system identifies the code lists, variables, data structure definitions and derived datasets affected by the change.

The third concerns guided curation using domain knowledge. A metadata curator activates a skill for the relevant statistical domain and reviews a variable or concept system with the model evaluating it against the conventions that the skill encodes.

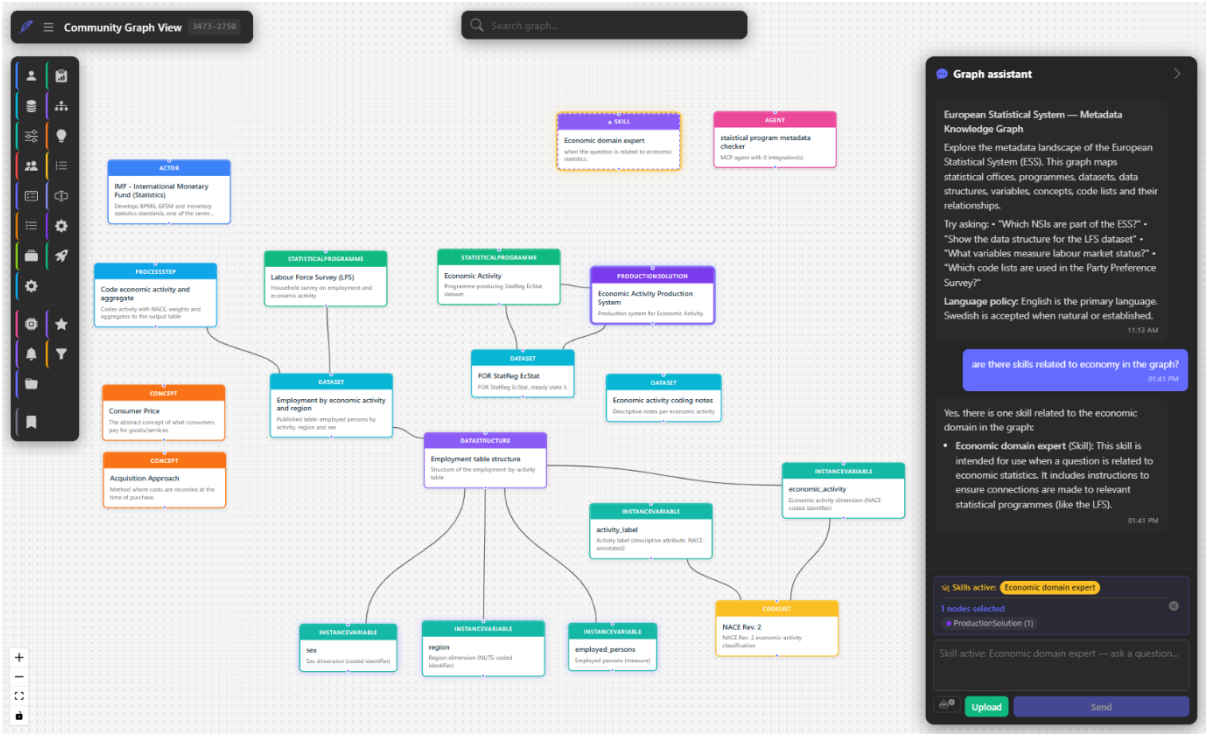
The fourth concerns dissemination. An officer explores the entities connected to a published data product and requests plain-language explanations suitable for publication, such as an account of what a variable measures or of which units a survey covers and which it excludes.

These user stories are supplied with the system, and each sets out a scenario together with the criteria by which its outcome would be judged. They therefore indicate the kind of work a reader launching the prototype could reasonably undertake with it, although they are written as descriptions of the use cases rather than as step-by-step instructions, and no guide exists for playing them through in sequence.

The system as configured for this domain demonstrates a populated graph of statistical metadata; natural language queries resolved against that graph; extraction of entities and relationships from uploaded documents; detection of overlapping definitions within the graph; expert agents carrying domain knowledge skills; and access for external agents over the published protocol.

Two capabilities exist in the platform but were not exercised in this work. The first is federation, by which metadata graphs held by different organisations can be connected. The second is the autonomous agentic loop described in Section 4, for which the structure exists but the implementation does not.

The Metadata Graph is published in its own repository, together with a guide to launching it in the shared environment. That guide is written as a sequence of steps: obtaining a key for the hosted language model, storing it as a secret, starting a development service with a pre-configured link that retrieves the code automatically, and starting the application with the profile for statistical metadata. The application then becomes available in the browser with the example metadata already loaded, so that a reader can begin exploring without preparing any data. The same repository holds the six user stories described above, one of which is written as an explicit validation exercise using a published labour force survey questionnaire; these are the most direct route into the system for a reader who wants to see what it does rather than how it is built. For an organisation wishing to go further, the repository also documents how a deployment profile is defined, which is what would be required to represent that organisation's own information model. As noted in Section 4, the application can be started without any language model configured, in which case the graph can be explored but the conversational features are absent.



[Figure 1: Screenshot from the Metadata Graph application]

5.3 News Corner: consistency between media reporting and official releases

Monitoring how official statistics are reported in the media is at present manual work, and the judgement required is genuinely difficult. An article and a release may share a subject and still differ in the figures quoted, the period referred to, the source credited or the emphasis given. A system that performs this task well must compare two texts, often written in different languages, along several dimensions at once, and must be specific about which dimension fails.

The multilingual aspect is not incidental. Statistical offices publish in their national languages while media coverage of their output frequently appears in English or in a third language, so the practical usefulness of this kind of analysis within the European Statistical System rests on the model being competent across a wide range of languages rather than in English alone. Testing that competence was one of the reasons for choosing this use case.

The experiment was deliberately kept simple. The intention was not to arrive at an architecture but to show how a test can be constructed that puts a language model directly against an operational need and returns an answer about what it can and cannot do. More elaborate arrangements can follow if continued development is judged worthwhile.

Articles were compared against official releases along five dimensions: whether they concern the same statistical subject; whether the figures reported are consistent with the release; whether the reference period matches; whether the official source is correctly credited; and whether the framing is faithful to the release rather than exaggerating or diminishing its findings. A test was considered passed when the model's overall judgement matched the expected one.

Seven tests were carried out. Six used the open-weights model available in the shared environment and one repeated the most difficult case using a commercial service for comparison. The material comprised releases and articles from the Dutch and Slovenian statistical systems, in English, Dutch and Slovenian.

Test	Model	Sources	Language	Expected	Result	Outcome
1	Open-weights	CBS and NL Times	English	Consistent	Consistent	Passed
2	Open-weights	CBS and NL Times	English	Consistent	Consistent	Passed
3	Open-weights	CBS and NL Times, different subject	English	Inconsistent	Inconsistent	Passed
4	Open-weights	CBS and NL Times, wrong figures and period	English	Inconsistent	Inconsistent	Passed
5	Open-weights	CBS release in Dutch, article in English	Dutch and English	Consistent	Consistent	Passed
6	Open-weights	SURS and RTVSLO, mismatched year	Slovenian	Inconsistent	Consistent	Failed
7	Commercial	The same case	Slovenian	Inconsistent	Inconsistent	Passed

[Table 1: Result table from the News Corner tests]

The sources named in the table are the statistical office of the Netherlands (CBS) and that of Slovenia (SURS), together with two media outlets: NL Times, an English-language news site covering the Netherlands, and RTVSLO, the Slovenian public broadcaster.

The model performed well given the difficulty of the task. It correctly identified consistent reporting, differences of subject, incorrect figures and incorrect reference months. It also handled correctly a comparison between a release published in Dutch and an article published in English, which is directly relevant in a system where offices publish in their national languages while coverage frequently appears in English.

The single failure is informative rather than disqualifying. The Slovenian article and release shared a subject and a quarter designation but referred to different years, and the model accepted them as consistent. The supplementary test established that the case was solvable, which locates the weakness precisely: the comparison of reference periods must be required to include the year and not only the period designation. This is a matter of how the model is instructed, and is addressable by design.

The tests were carried out manually, but the work also produced a specification for the operational system that such a capability would support. The specified system collects feeds from news outlets and from statistical offices, stores their content locally, generates candidate pairings between articles and releases, submits those pairings to a language model for comparison, and produces alignment scores together with reporting and a benchmarking function.

Two principles in that specification are worth carrying into other work of this kind. The language model is used only where interpretation of meaning is required, namely identifying the statistical subject, extracting the claims made, comparing article with release and producing the final assessment; the collection, parsing, deduplication, scheduling, storage and aggregation of scores are performed by ordinary deterministic software. And the number of calls to the model is minimised by design, since content is stored locally and candidate pairings are generated deterministically before any call is made, so cost scales with the number of genuinely new comparisons rather than with the volume of material collected.

This contribution is an experiment rather than a runnable system. The test set is small: six tests with the open model and one comparison. That is sufficient to establish feasibility and to locate one

specific weakness, but not to support claims about reliability across languages, subjects or statistical domains. No reproducible implementation was produced, so the results cannot at present be re-executed by a third party. This is the principal difference between the experiment and a prototype. A reader wishing to take this work further will find in the work package repository the full test report, including the individual results for each of the five dimensions and the instruction used to elicit the model's judgement, together with the architecture specification for the operational system. Together these are sufficient for another organisation to repeat the tests on its own material or to implement the specified system.

6. Evaluation results

The table below records the assessment of each contribution against the common dimensions. It concerns the first of the two evaluation questions set out in Section 3, namely whether each system is reasonable and operable, and not the second, namely how accurate its output is.

Dimension	Web Corner	Metadata Graph	News Corner
Efficiency gain	High in general applicability, low in execution speed	High for discoverability and cross-organisation collaboration, low as a replacement for authoritative registers	High compared with manual work
Reusability	High, reusable across sites without new code	High, the profile mechanism supports any metadata domain without code changes	High by intent, though no runnable implementation yet exists
Data accessibility	High, public pages without credentials, subject to the considerations on collection practice in Section 5.1	Medium, example data is self-contained but real use requires internal registers	Medium to low, owing to paywalled news content
On-premises compatibility	Medium to high, requires local capacity to operate a model supporting tool calling	Medium to high, no external dependency and operable without a model at all, but useful behaviour requires local capacity	High, requires only a conversational endpoint with no tool calling
Adoption effort for offices	Medium to high, small system but some technical setup required	Medium, launching and exploring is documented, populating with real metadata is not trivial	Medium to low
Evaluation robustness	Low, assessed by inspection with no reference data	Low to medium, validation scenarios exist but no scored benchmark	High by design, the benchmark method is specified but not yet executed at scale
Feasibility	Medium	Medium, example data demonstrates the concept, real coverage requires curation	High, demonstrated by the tests
Lifespan	Medium to high, resilience to site redesign is the central argument	Medium to high, standards-aligned schema and protocol-based integration reduce lock-in	Medium, owing to rapid change in media
Cost effectiveness	Medium to high, a completed task costs a few cents at commercial rates	Medium to high, cost is incurred per question rather than per record; curation effort dominates	High compared with manual work

[Table 2: Evaluation results - Dimensions and prototypes]

7. Scope and limitations

All three contributions are proof-of-concept work and are not production-ready. They are also at different levels of maturity: two are runnable systems and one is an experiment with a designed but unimplemented architecture.

Results are dependent on the model used. Changing provider, version or configuration may produce different outcomes, and the output of a language model is not deterministic, so the same input may yield results of differing quality across runs.

Several limitations concern data and access. The material used in some of the use cases is perfectly usable for the purpose, but in most cases carries restrictions on further distribution, and it can therefore not be included as part of the prototypes themselves. Language coverage is uneven, with English performing best and other European languages showing more variable quality. Access to internal metadata and production systems was not available, so the metadata work is based on curated public example data rather than on an organisation's real holdings. Collection from the web is constrained in practice by site terms of use, access restrictions and automated-access controls rather than by the availability of the material itself. Above all, the shared environment permits only non-sensitive material, so every use case was exercised on public or otherwise non-sensitive data.

Technically, on-premises deployment of open models supporting tool calling remains demanding for offices without local capacity to operate a model of moderate size. Offices also differ considerably in the extent to which they may use cloud services to obtain access to models, whether open or commercial. That possibility often depends on the data in question rather than on the technology: metadata may in many cases be regarded as non-sensitive and therefore manageable in an external service, whereas use cases involving more sensitive material will leave many offices with very limited scope for using external cloud services at all. The latency and cost of model calls may constrain deployment at operational scale, and agentic loops multiply the number of calls required for a single task. Content that is assembled dynamically when a page is displayed is not yet handled reliably by the collection prototype. The metadata system requires curation effort to populate with real material. Autonomous agentic loops are not implemented in either prototype.

Methodologically, the evaluation relies in part on the subjective judgement of the participants, and its criteria are an indication of what appears to matter rather than a settled framework. Systematic evaluation of effectiveness was outside the preconditions of the work for the reasons given in Section 3. No longitudinal evaluation has been carried out to assess how output changes as models are updated. Consequently this report evaluates whether the systems are reasonable and reusable, not how accurate they are, and statements about output quality should be read as observations from individual cases rather than as measured performance.

8. Observations on the quality of generative approaches

The clearest observation from this work is that the open models available in the shared environment performed well across several of the systems and examples developed. Consistency checking of media reporting, autonomous navigation of websites and grounded conversation over a metadata graph were all carried out with the same open-weights model, without recourse to a commercial service.

This matters more than any individual result, because it changes what an office must assume before starting. Applying comparable solutions in these areas does not require a commercial model, nor a decision to send content to an external provider. It requires the capacity to operate an open model of moderate size, which is a considerably more tractable prerequisite and one that several offices can already meet. Multilingual work forms part of this observation, since a release published in Dutch was compared correctly against an article published in English.

Differences nonetheless remain where a task demands more advanced reasoning, or where an agentic workflow requires sustained judgement across several steps. The clearest instance is the reference period case described in Section 5.3, where an article and a release shared a subject and a quarter designation but referred to different years. The open model accepted them as consistent while a commercial model did not. The difference was resolved by requiring the full reference period to be extracted explicitly before any judgement was made, which is the useful form of such a finding: the weakness appeared in a specific and identifiable kind of reasoning and proved addressable by design. Agentic workflows show the same pattern in another form, in that enabling extended reasoning materially improves the quality of the output, and the strategy encoded in the instructions performs much of the work that would otherwise require a stronger model. Both come at a cost in latency, in tokens consumed and in the effort spent on designing the instructions.

A further observation concerns the ecosystem rather than the models. The components available for building agentic solutions have developed substantially since the previous deliverable. Tool calling is no longer something to be demonstrated in principle but was applied in working systems as the means by which a model drives retrieval and traversal rather than merely interpreting material placed before it. The Model Context Protocol was likewise applied in practice, making a statistical system reachable by external agents over a standard protocol rather than through bespoke integration. And the packaging of domain knowledge as loadable skills has emerged as a further usable technique, allowing expertise to be supplied, and an agent's scope of action constrained, as configuration rather than as code. The practical consequence is that the building blocks for such solutions are now available rather than needing to be devised for each project, and the binding constraint has moved from whether such systems can be built to how well they perform and how their use is governed.

Several practical recommendations follow. Instructions to the model should be structured and explicit about the steps required, since this is where open models most often close the gap on the harder cases. Benchmark datasets should be established so that quality can be tracked over time and across changes of model. Comparison between models is worth including in the evaluation workflow, following the pattern used in the News Corner tests, where a case that one model failed was repeated on a stronger model in order to distinguish a limitation of the model from an unsolvable task. Operations offered to a model should be kept narrow and well defined, since this reduces the number of points at which an agentic arrangement can fail. And where a capability may later be needed by other systems, it is worth exposing it through a standard protocol rather than only through an interface specific to the application.

9. Future work

The directions set out below describe possible ways forward, given the prototypes now delivered and the experience built up in the course of this work. They are not a plan of record. Some of them could be pursued within the AIML4OS project itself, where this work package and its neighbours are

the natural setting. Others are better suited to being taken up outside it, by an organisation that starts from these prototypes and develops the underlying structures further for a use case of its own, or within projects that individual national statistical institutes may choose to run. Which route is appropriate differs from item to item, and for the more substantial items it is unlikely to be realistic to carry them through within the remaining scope of the work package alone. The section should therefore be read as an indication of where continued effort would be most useful, and of the forms that effort might take.

The most valuable continuation concerns evaluation, which is the second of the two questions set out in Section 3 and the one this work could not reach. The prototypes are the means of reaching it, and they could serve as the basis for a systematic assessment of effectiveness carried out in environments cleared for sensitive material, where an organisation's own data can supply the reference against which output is judged. Establishing benchmark datasets for each use case would allow models to be compared systematically, against a fixed version and repeated as models change. The benchmark method described for News Corner, in which articles of known alignment are scored by human evaluators and the system's agreement with that reference is measured, offers a template that could be applied to the other use cases. Workflows involving human review would support continuing quality assessment, and comparison against rule-based baselines and existing tools would place the results in context. This is work that the European Statistical System is better placed to undertake collectively than any single office acting alone.

On the systems themselves, News Corner would need to be developed from an experiment into a runnable implementation before it could be used operationally. The handling of dynamically assembled web content remains to be completed if collection is to be reliable on the most modern sites. The metadata graph would benefit from being populated with real material, and federation across organisations remains untested. The autonomous agentic loop is the natural next architectural step, and the assessment of the consequences of classification change is the use case that most requires it.

On deployment and reuse, packaging the systems for straightforward deployment by other offices would lower the threshold for adoption, as would testing on-premises operation with open models on behalf of offices working under data protection constraints, documenting the infrastructure genuinely required to operate a model supporting tool calling, and providing lighter-weight guidance for less technical users.

On integration, the relationship to existing statistical production systems and metadata registers is largely unexplored, as are the ways in which output from such systems might feed into quality assurance and dissemination workflows and align with established process models. The Model Context Protocol merits examination as a general integration pattern for making statistical systems available to agents, together with the governance questions that follow from it.

Finally, the fine-tuning of open models for statistical tasks, multi-agent architectures for complex workflows, and the contribution of retrieval-augmented generation to output quality remain open research directions. The first of these falls within the scope of Task 12.3 and will be addressed there.

10. Conclusions

The work reported here produced two runnable prototypes and one experiment, satisfying the requirement under Task 12.2 of at least two prototypes across at least two high-value areas, namely

the handling of data and metadata and the analysis of large documents and web page data. Four conclusions follow.

The first is that this work moves the work package from treating language models as processors of text to treating them as participants in a process. In the earlier deliverable the prototypes used models to interpret content that conventional software had already retrieved; here, both prototypes allow the model to decide what to retrieve or which relationships to follow. The practical consequence is that capability which previously required code written for each site or source is now carried by a set of instructions and a small number of narrow operations, which moves the maintenance burden from many fragile scripts to a single set of instructions and a single layer of operations. For statistical production, where sources change continuously and maintenance dominates the total cost of a collection, this is the most consequential property observed. What made the shift possible within the available time was not only the capability of the models but the maturing of the surrounding ecosystem described in Section 8.

The second is that an open model operating on infrastructure that a statistical office could realistically provide proved sufficient for all three use cases at the prototype level. The same model handled multilingual comparison of statistical releases, autonomous multi-step navigation of websites, and grounded conversation over a metadata graph. It was outperformed by a commercial model on the single most difficult case, and the manner of that difference was instructive, since it was resolved by revising the instructions rather than by changing model. The practical conclusion is that the capability of the model is not the binding constraint at this level of ambition; the capacity to operate a model supporting tool calling is. Because the environment used admits only non-sensitive material, this says nothing about the processing of sensitive data, but it does establish that the arrangement an office would need to reproduce for that purpose is an ordinary one, and that choosing an open model over a proprietary one does not cost capability in these applications.

The third is that the manner in which a capability is made available matters as much as the capability itself. The two prototypes apply tool calling in structurally different ways, as a private control loop within an agent and as a published interface over a standard protocol, and the difference determines what can be built next. By offering its operations both to people through a browser and to agents over a standard protocol, backed by a single underlying service, the Metadata Graph ceases to be an application with an artificial intelligence feature and becomes a source that other agents can reason over. This is a general pattern rather than one specific to metadata, and it is the most transferable architectural result of the work. It also raises governance questions that the work package has not yet addressed: when an agent can query and modify a statistical system through a standard protocol, the boundaries of what it may see and change become an explicit design decision rather than an implicit consequence of the user interface.

The fourth is that the prototypes answer the first evaluation question and set up the second. Whether these systems are reasonable, buildable, reusable and operable under realistic conditions is the question this deliverable answers, and the answer across all three use cases is affirmative. How accurate and reliable their output is, measured systematically against a known correct answer, was outside the preconditions of the work. That describes where the work now stands and identifies what the prototypes are most useful for next: they are working, documented systems that can serve as the basis for systematic assessment of effectiveness, carried out where sensitive material may also be used and where an organisation's own data supplies the reference that public sources cannot. Applying such a design to each use case, against a fixed model version and repeated as models change, is the natural continuation, and it is work that the European Statistical System is better placed to undertake collectively than each office separately.

Taken together, the results indicate that the technical barriers to the use of language models in official statistics are now lower than the methodological ones. Building a working agentic prototype is a matter of days. Establishing how well it works, reliably and repeatably, is the next piece of work, and it now has something concrete on which to be performed.

Supporting material and information

The prototypes and the detailed reports behind this document are published in two public repositories. The work package repository holds the Web Corner prototype together with its own report and the full transcripts of the runs described in Section 5.1, the report on the Metadata Graph, and the three News Corner documents: the summary of the experiment, the full test report and the architecture specification for the operational system. The Metadata Graph application itself, together with its user stories and the guide to launching it in the shared environment, is published in a repository of its own. The shared development and hosting environment referred to throughout this report is operated as a service to statistical organisations and is reached through its own portal.

The work package repository is the starting point, at https://github.com/AIML4OS/WP12. Within it, the Web Corner prototype, its report and the run transcripts are held at https://github.com/AIML4OS/WP12/tree/main/stockholm_sprint_2026/webcorner, the report on the Metadata Graph at https://github.com/AIML4OS/WP12/tree/main/stockholm_sprint_2026/metadata, and the News Corner material at https://github.com/AIML4OS/WP12/tree/main/stockholm_sprint_2026/News_Corner. The Metadata Graph application is published separately at https://github.com/AIML4OS/WP12_MetadataGraph, where the guide to launching it in the shared environment and the user stories referred to in Section 5.2 are also held.